

MODULE 5

Introduction to Web drivers: First Steps, what is WebDriver? How WebDriver works? The history of Selenium, the tools you need to get started.

Locating elements on a page: Locating one or more elements based on ID, class, or name, Fine-grained targeting using complex CSS selectors, making pages amenable to element locating.

What is Selenium WebDriver?

Selenium WebDriver is a robust open-source framework for automating web browsers, primarily aimed at easing the testing and verification of web applications. As an important part of the Selenium suite, WebDriver offers a programming interface to interact with web browsers, allowing developers and testers to automate browser actions seamlessly. Unlike its predecessor, Selenium RC (Remote Control), WebDriver directly communicates with the browser, providing a more stable and efficient means of automation.

1. WebDriver supports various programming languages, including Java, Python, C#, and JavaScript, making it adaptable for developers working in different technology stacks.
2. It allows the automation of diverse tasks such as navigating web pages, interacting with web elements, submitting forms, and validating expected outcomes.
3. WebDriver's cross-browser compatibility ensures that tests can be conducted across different browsers like Chrome, Firefox, Safari, and Internet Explorer, promoting consistent behavior across various platforms.
4. The framework's flexibility, coupled with an extensive community and active development, positions Selenium WebDriver as a cornerstone in the field of web automation and testing. Its capabilities extend beyond testing, as WebDriver is often used for web scraping, data extraction, and other browser automation tasks in diverse software development scenarios.

Selenium WebDriver is a powerful tool used for automating web browsers by simulating user interactions such as opening a website, clicking on links, filling out forms, and navigating through pages. It is widely used for automated web testing as it helps save time and reduces the cost associated with manual testing.

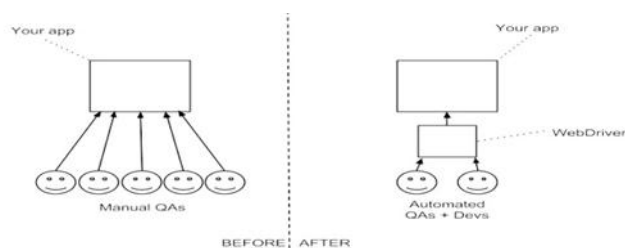


Figure 1. Before WebDriver

Benefits of Using WebDriver:

- **Multi-browser support:** WebDriver works with all major browsers like Chrome, Firefox, Safari, and more, providing flexibility in testing across different platforms.
- **Supports multiple programming languages:** You can write your test scripts in various programming languages such as Java, Python, C#, Ruby, and others.
- **Mimics real user actions:** WebDriver performs browser actions using native events, making interactions more realistic compared to older Selenium versions.
- **Open-source:** Selenium WebDriver is free to use and has strong support from a vast community of developers.
- **Cross-platform compatibility:** It operates on major operating systems, including Windows, Linux, and macOS, as well as mobile platforms like Android and iOS.

Standardized: WebDriver has become a W3C standard, ensuring consistent behavior across different browsers and versions.

- **No legacy issues:** Unlike Selenium RC, WebDriver does not have the limitations and problems that affected older Selenium tools.
- **Simpler syntax:** WebDriver has a more intuitive and easier-to-use syntax than Selenium RC, which makes it a popular choice for automation.

What is Need of Selenium WebDriver?

To get why WebDriver was a big deal, we must look at the issues with its old version, Selenium RC.

Selenium Remote Control (RC) was a tool for testing that let coders create automatic UI tests for web apps in any code language. It could test websites over HTTP in browsers that could run JavaScript, the old Selenium RC Server took commands from your test code, made sense of them, and sent back results. It did this by putting Selenium core into the browser to run commands. This method was complex and slow.

Selenium WebDriver fixed this by dropping the need for a separate server. It talks straight to browsers, using their own built-in ways to automate tasks. This simpler setup cuts down on run time.

WebDriver gives clear APIs, not like the tricky ones from RC. Plus, it can run tests without showing the browser, using the GUI-less HtmlUnit browser. These upgrades make WebDriver easier to use and faster than the old way.

How WebDriver works?

WebDriver is a powerful tool that works with all major web browsers and supports various programming languages for automated testing. It achieves this through a combination of components: the web browser itself, a plugin or extension within the browser that implements the WebDriver JSON API, and language bindings (e.g., Java) that make HTTP requests to this API. When a test script runs, it opens the browser and activates the plugin, allowing the script to send requests to perform actions like clicking links or typing text.

The plugin acts as a server, receiving commands and translating them into browser actions. Since the plugin only needs to implement the JSON API, it can be created for any major browser, ensuring compatibility. This structure also means that WebDriver's language bindings can communicate with the browser regardless of the programming language being used, making it a flexible solution for automated web testing.

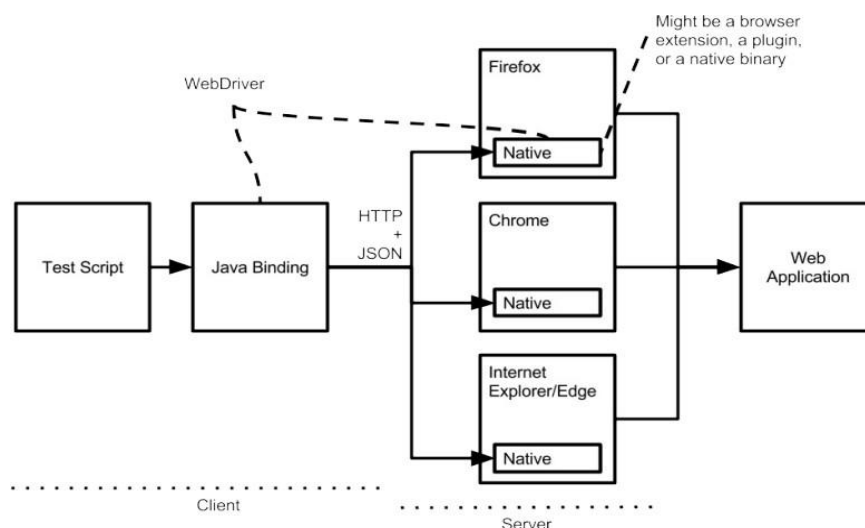


Figure 2. Web driver diagram

Limitations of WebDriver:

- **Timing issues:** WebDriver does not automatically handle when elements appear on a web page, so you need to code wait mechanisms to deal with such timing issues.
- **No built-in change detection:** WebDriver cannot tell when changes occur on a page, so you cannot rely on it to detect updates automatically.

Limited built-in utilities: WebDriver does not provide many utilities for coding, so developers often need to write custom solutions for various tasks.

- **Handling complex page elements:** It does not natively support elements

composed of multiple components, like JavaScript-based date pickers or dynamic pop-ups.

- **No built-in framework:** Selenium WebDriver does not come with an integrated testing framework, so it is recommended to use frameworks like JUnit or TestNG for structured test development.
- **Browser management:** WebDriver does not handle browser instances automatically; you need to manage the browser's lifecycle and clean up after tests.
- **No automatic browser installation:** Unlike some tools, WebDriver does not install or update browsers for you, so you need to set up and maintain them manually.

History of Selenium:

The history of Selenium begins with **Selenium Core**, which was created by Jason Huggins at ThoughtWorks in Chicago. Selenium Core was built to automate user interactions with Internet Explorer and didn't require an additional graphical interface tool—only Java, which most developers already had installed on their systems.

Later, **Shinya Kasatani** developed **Selenium IDE** as a Firefox plugin based on Selenium Core. Selenium IDE provided a user-friendly graphical interface that allowed users to record and replay browser actions. This feature made it popular among quality assurance (QA) professionals and business analysts who did not have programming skills.

Selenium RC (Remote Control) came next and was an enhancement of Selenium Core, designed to allow tests to run across multiple machines, which improved test scalability. **Selenium WebDriver** was introduced by Simon Stewart and first presented at the Google Test Automation Conference. It offered a more straightforward and effective API compared to Selenium RC and was later included in **Selenium 2.0** in 2008, marking a significant shift in how automated web testing was performed.

Why the Name "Selenium"? The name "Selenium" came from a joke by Jason Huggins. He referenced a competitor called Mercury, noting that mercury poisoning could be treated with selenium supplements, and humorously suggested that his tool could "cure" the problem with Mercury, hence the name "Selenium."

Who is a Selenium Tester?

The work of a Selenium Tester depends on the role, company to another, from one project to another, and even also varies according to a year of experience. There are some standard responsibilities of a Selenium Tester.

Understand the requirements and functionality of the client/user.

Knowledge of Regression and Functional Testing.

Knowledge of the Selenium suite of tools like Selenium IDE, Selenium Grid, and Selenium WebDriver.

Test Automation Framework.

Create Test cases and able to execute them.

To fix errors, resolve queries if any also debug the test cases.

If you want or thinking of pursuing a career in the web testing field, then there are various Selenium career opportunities. Selenium helps developers with automating testing.

There are many Career Opportunities as a Selenium Developer:

Automation Test Engineer

QA Manager/REMOTE – (Automation/Selenium)

SDET / QA Engineer

Quality Assurance Selenium remote

Senior Java SDET

Test Engineer

Selenium Automation Lead

QA Analyst

You must have lots of questions pop up How to start? Where to start? What do we need to learn before beginning Selenium? In this article, we will discuss all these things in detail.

Roadmap to Learn Selenium

Learning Selenium is not that much difficult, it requires a good command in any of the programming languages and practice properly. So, To understand or before starting to learn Selenium you should have some good commands on any programming language, HTML, Unit testing, TestNg, and Framework to learn Selenium for Automation testing. Let's discuss all these in more detail.

Step 1: Learn Any Programming Language - you should know basic knowledge about any programming language. Taking Java as an example. First, you should have a proper installation and setting up Eclipse for practice.

The first thing you need to learn is Core Java. So, you need to learn the following major

topics:

Class and Object

Keywords, Variables, Identifiers, and Datatypes

Operators

Type Conversion

Selection Control Structures - if, switch

Iteration Control Structures

Methods

Constructors

Memory Management

OOPS Concepts (Inheritance, Encapsulation, Abstraction, Polymorphism)

Access Modifiers

Class Diagram

String & Array

Object and Wrapper Classes

Exceptions

Some Advance Java concepts of Java need to cover to be aware of all these concepts along with basic concepts

Collection Framework - A framework is a set of classes and interfaces which provide a ready-made architecture. To implement a new feature or a class, there is no need to define a framework.

Local Date and Time - LocalDateTime in Java is used for displaying the local date and time on the output screen.

Lambda Expressions and Stream - Java 8 introduced a host of new features, including lambda functions and streams.

JDBC (Java Database Connectivity) - Java Database Connectivity is a Java API to connect and execute the query with the database

Step 2: IDE and Build Tools - Once you cover all the core and advanced java and start to practice therefore you must know the following IDE's

IntelliJ IDEA

Eclipse

Now all your java code is built. So, for building your Java project you must be aware of Maven (build tool).

Step 3: Learn DataBase Management System - Knowledge of Database Management System (DBMS) is good to go. Listed down all the important topics which need to cover before starting with Selenium

Introduction to DBMS, Keys, and ER Diagram

SQL Basics

Normalization

Functional Dependency

DDL and DML Statements

SQL Functions, Sorting

Updating Data, Deleting Data, Case, Order By

Grouping, Combining Data, and Cartesian Product

Joins

Step 4: Learn Fundamentals of HTML - You should have basic knowledge and some practice, as while working with selenium You would need to find out locators of web elements like radio, links, text boxes, checkboxes, etc.

Basic HTML Element

Form with Table Element

Anchor Element

Media Element

Audio and Video Element

Text Formatting Element

Step 5: Know about Testing Basics and Testing Management - Knowledge about basic concepts of software testing that are to be understood by anyone who is going to work on a software testing project. List down some concepts

The need for Software Testing

Basics Concepts and Terminologies

How Software Testing is Done

Requirement Analysis Phase

Test Planning and Design Phase

Test Execution Phase

Test Closure Phase

Step 6: Developing Persistence Layer of Enterprise Application using Hibernate - Hibernate Framework is a very popular persistence layer framework for developing Java enterprise applications.

Hibernate is a Java-based, open-source, Object Relational Mapping (ORM) framework. About Persistence Layer

Object Rational Mapping

Performing Hibernate CRUD (Create, Read, Update, Delete)

Criteria of API (Application Program Interface)

Step 7: Start with Selenium - Selenium is a suite of tools that can automate actions on web browsers. Each tool is developed to support a specific kind of automation approach. For more information do read Selenium Basics – Components, Features, Uses, and Limitations

There important tools that form the core of the Selenium ecosystem are:

Selenium IDE

Selenium WebDriver

Selenium Grid

Step 8: Selenium Basics - Start with learning Locators and WebDrivers.

There are many different ways to find web elements like ID, Name, Class Name, Link text, Partial Link text, CSS, and XPath.

You have to learn commands on WebDrivers and their real-time knowledge. Some commonly used WebDriver Like Google Chrome Driver, Internet Explorer Driver,

Opera Driver, Safari Driver, and HTML Unit Driver

You can first target the simple things to learn Selenium Basics:

WebDriver and Locators

Browser Commands

Get Commands

Browser Navigation Commands

Web Element Commands

Select Dropdown Commands

Actions Class

Selenium Waits

Take a Screenshot in Selenium

Handle Alerts or Pop-ups

Step 9: Testing - Testing is the process of verifying and validating the functionality of an application to make sure that it works as per the requirements and does not have a bug/error. It is the responsibility of the developer to test the module or method that he/she has developed before the code is integrated with others' code.

The most commonly used unit testing frameworks are JUnit and TestNG.

Step 10: Selenium Advance - Now move to learn advanced concepts to be better commanding in Selenium Framework. Learn the following components which will help you in creating a maintainable framework/script in Selenium.

i) Work with TestNG Framework - TestNG (Test Next Generation) is an open-source automated testing framework. It is similar to JUnit, but it is not a JUnit extension.

Installation

Annotations

TestNG Assertions

ii) Enhance Test Scripts -

XPath axes - An XPath expression is an alphanumeric pattern that returns a node or a set of nodes in the given XML/HTML document.

SSL (Secure Sockets Layer) - SSL is a standard security protocol that is used to establish a secured connection between the server and the client end.

JDBC (Java DataBase Connectivity)

iii) Log Reports using Log 4j - Apache Log 4j is a Java-based logging utility that is used for logging execution information.

Logger

Layout

Appender

iv) Build Automation using Maven - Working with Maven is to build the project more automated and Independent.

v) Build triggers using Jenkins - Jenkins is an open-source continuous integration tool used on Windows, Linux, and Mac OS environments.

vi) Automation Framework - An automation framework is a combination of various guidelines, coding standards, project hierarchies, modularity, reporting mechanism, test data injections, etc. to pillar automation testing.

Some concepts in the Automation Framework

Create Test Cases

Create and Execute Test Batches

Tracking and Reporting Errors/Defects

Regression Testing

vii) Cross-browser testing using Selenium Grid - Selenium grid is part of the selenium suite which distributes the test across multiple physical or virtual machines for the test scripts can be executed in parallel (Simultaneously). It helps us in covering the test process across browsers and different platforms.

Step 11: Practice - You should keep practicing all the above topics to understand the real-time implementation of selenium.

Some Selenium Practice Exercise

Automate Google Search with Selenium.

Login to Email using username and password automatically.

Send an Email Automatically using Selenium.

Automate Demo Web Table with Selenium.

Tools You Need to Get Started:

1. Java Development Kit (JDK):

- Java is one of the most popular and widely used programming languages, and it will be used throughout this book. Java 8 or newer is recommended as it includes modern features such as streams and lambda expressions that enhance coding efficiency.
You can check if Java is installed and find its version by running `javac -version` in the terminal.
- If you need to install it, Linux users can use their package manager (e.g., `apt` or `yum`), while Windows and macOS users can download it directly from Oracle's website.

2. Apache Maven:

- Maven is a build tool used to manage project dependencies and streamline the build process. It simplifies handling libraries and external tools required for Selenium testing.
- To verify if Maven is installed, use the `mvn -version` command in the terminal.
- Linux users can install Maven with commands like `sudo apt-get install maven` or `sudo yum install maven`. macOS users can install it using Homebrew (`brew install maven`). Windows users can download it from the [Apache Maven website](#).

3. Google Chrome:

- Chrome is the most supported browser for Selenium due to its cross-platform availability and simple integration with WebDriver.
- Linux users can install Chrome using a package manager, while others can download it from the [Chrome website](#).

4. Git:

- Git is a version control system needed to access source code for this book and other projects.

- On macOS, you can use Homebrew (brew install git), and on Linux, you can use sudo apt-get install git.
- Windows users can download Git from git-scm.com.

To start using Selenium WebDriver, ensure that you have Java, Maven, Chrome, and Git installed on your system. Selenium WebDriver is an efficient tool for automating web browser interactions, making it an essential part of modern web testing. While it has some limitations, such as the need for custom code and manual browser management, its benefits far outweigh the drawbacks for most use cases.

Features of Selenium WebDriver

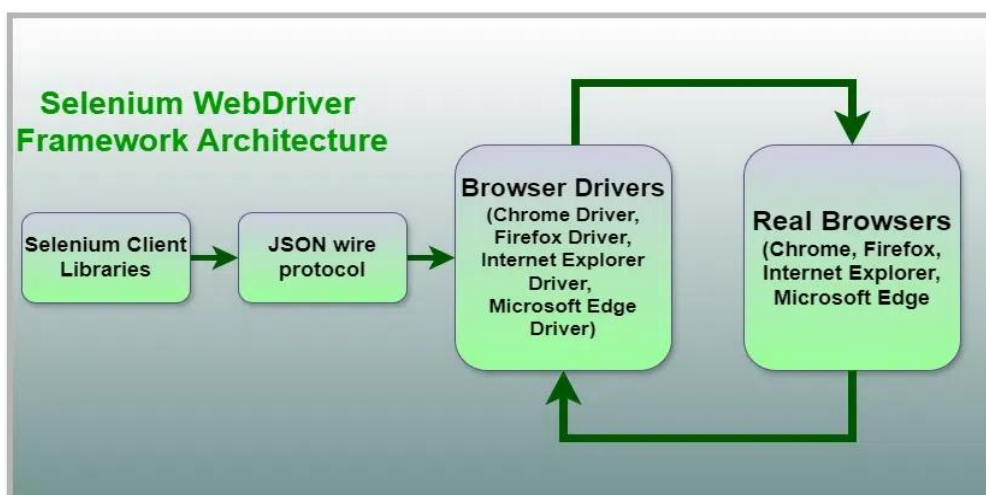
Direct Communication with Browsers: Unlike Selenium RC, WebDriver interacts directly with the browser's native support for automation, leading to more stable and reliable testing. This direct communication contributes to improved performance and better handling of complex web page interactions.

Support for Parallel Execution: WebDriver allows for parallel test execution, enabling faster test cycles and efficient utilization of resources. This is particularly useful in large-scale testing environments where multiple tests can run simultaneously.

Rich Set of APIs: WebDriver provides a comprehensive set of APIs for navigating through web pages, interacting with web elements, handling alerts, managing windows, and more. This richness in APIs empowers testers to simulate real user interactions effectively.

Selenium WebDriver Architecture

The Selenium WebDriver Architecture has several components that work together to automate the web browsers.



1. Selenium Client Libraries:

Selenium supports various programming languages such as Java, Python, C#, Ruby, and more. These libraries provide bindings or APIs that allow you to interact with Selenium and control the browser using the chosen programming language.

For example, if you are using Java, you would use the Selenium Java client library, and if you are using Python, you would use the Selenium Python client library.

2. JSON Wire Protocol:

JSON Wire Protocol is a RESTful web service that acts as a communication bridge between the Selenium Client Libraries and the Browser Drivers.

It defines a standard way for sending commands to the browser and receiving responses. These commands include actions like clicking a button, filling a form, navigating to a URL, etc.

The protocol uses JSON (JavaScript Object Notation) as the data interchange format for communication between the client and the server (browser).

3. Browser Drivers:

Browser Drivers are executable files or libraries specific to each browser (ChromeDriver for Chrome, GeckoDriver for Firefox, etc.).

They act as intermediaries between the Selenium Client Libraries and the actual browsers. The client libraries communicate with the browser drivers, and the drivers, in turn, control the respective browsers.

The browser drivers interpret the commands from the Selenium Client Libraries and convert them into browser-specific actions. They also send information back to the client libraries about the status of the commands executed.

4. Real Browsers:

Real Browsers are the actual web browsers like Chrome, Firefox, Safari, etc.

The browser drivers launch and control these real browsers based on the commands received from the Selenium Client Libraries. The browser drivers establish a communication channel with the browsers to automate user interactions.

The real browsers execute the commands, perform actions on web pages, and return the results to the browser drivers, which then pass the information back to the Selenium Client Libraries.

Advantages of Selenium WebDriver

Cross-Browser Compatibility: Selenium WebDriver allows you to execute tests across different web browsers such as Chrome, Firefox, Safari, Internet Explorer, and others. This ensures that your web application is compatible with a variety of browsers, providing a more reliable assessment of its functionality.

Multi-language Support: Selenium WebDriver supports multiple programming languages like Java, Python, C#, Ruby, and more. This flexibility allows QA engineers and developers to choose a language they are comfortable with or that is best suited for their project.

Cost-Effective: Automated testing with Selenium WebDriver reduces the need for manual testing, saving time and resources. Automated tests can be run repeatedly without incurring additional costs, making it a cost-effective solution in the long run.

No Need for Remote Server: Selenium WebDriver doesn't require a remote server for communication with browsers. Direct communication between the WebDriver and the browser eliminates the need for a separate Selenium server, simplifying the test setup.

Supports Multiple Operating Systems: Selenium WebDriver is compatible with various operating systems, including Windows, Mac, Linux, etc. This cross-platform support allows teams to execute tests on different operating systems, ensuring the application's consistency across diverse environments.

Introduction to Locating Elements

WebDriver provides several ways to locate elements on a webpage. Locating elements is an essential part of web automation as it allows interaction with specific parts of a webpage. All locating operations occur within a **search context**, which defines the scope for finding elements.

What is a Search Context?

A **search context** is the boundary or area where elements can be searched.

- **SearchContext**

Interface: public

interface

SearchContext {

List<WebElement> findElements(By by);

WebElement findElement(By by);

}

- findElements(By by): Returns all elements matching the given locator. If none are found, it returns an empty list.
- findElement(By by): Returns the first element matching the locator. Throws NoSuchElementException if no matching element is found.
- Both the **WebDriver** and **WebElement** interfaces extend the **SearchContext**, enabling these methods to work seamlessly.

What is a Locator?

- A **locator** is a strategy used to identify elements on a page.
- Locators in Java are created using the By class.
- Example:
 - Find an element using the tagName:

```
WebElement heading1Element = driver.findElement(By.tagName("h1"));
```

- Find all elements using the tagName:

```
List<WebElement> paragraphElements = driver.findElements(By.tagName("p"));
```

Types of Locators in WebDriver

1. Class Name

- Locates elements by the value of their class attribute.
- Example:

```
driver.findElement(By.className  
e("btn"));
```

- **Note:** Avoid using this for unique element identification as many elements may share the same class.

2. CSS Selector

- Uses CSS selectors to locate elements.

Example:

```
driver.findElement(By.cssSelector("div.container > p"));
```

1. ID

- Locates elements by their id attribute, which is often unique.
- Example:

```
driver.findElement(By.id("change-  
password"));
```

- **Best Practice:** Use this when available, as IDs are usually unique.

2. Link Text

- Locates a link by the exact text displayed.
- Example:

```
driver.findElement(By.linkText("Forgotten
```

Password"));

3. Partial Link Text

- Locates a link by a partial match of its text.
- Example:

```
driver.findElement(By.partialLinkText("Forgotten"));
```

- **Caution:** Can result in multiple matches if other elements contain the same partial text.

4. Name

- Locates elements by the name attribute, typically used for form inputs.
- Example:

```
driver.findElement(By.name("email"));
```

5. Tag Name

- Locates elements by their HTML tag name.
- Example:

```
driver.findElement(By.tagName("h1"));
```

- **Use Case:** Effective for headings and groups of elements.

6. XPath

- Uses XPath expressions to locate elements, providing flexibility for complex structures.
- Example:

```
driver.findElement(By.xpath("//input[@mail]"));
```

What are Locating Strategies in Selenium?

Selenium Locating strategies are a set of methods that we can use to precisely locate the target web element on a webpage. Locating the target web element is a crucial step in automating user interactions on a web page. Selenium offers 7 locating strategies that we can use to locate the target web element precisely on a web page.

1. Locating By ID

The ID attribute of an HTML element is a unique identifier, the id attribute is assigned to a specific element and should be unique on a web page and this makes it an ideal choice for locating an element because it guarantees that you're targeting the exact element you intend to interact with.

Searching for an element using its ID is generally faster than other methods such as XPATH or CSS Selector because it leverages native browser functions optimized for this purpose.

To locate an element by ID we use By.ID locator provided by the By class.

By is a class that provides a set of locating strategies to locate web elements on a web page.

Syntax:

```
element=driver.find_element(By.ID, "element_id")
```

```
# Python program to implement Locating by ID
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url
```

```
url = "https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/"
```

```
driver.get(url)
```

```
# Replace ID_NAME with the Id of your element
```

```
try:
```

```
    # Replace this with the id of the element
```

```
    element = driver.find_element(By.ID, "nav_tab_courses")
```

```
except NoSuchElementException:
```

```
    print("Not Found")
```

```
else:
```

```
    print("Found")
```

2. Locating By Class Name

The class attribute is one of the most commonly used in attribute in HTML .The class attribute allows multiple elements on a webpage to share a common styling. While classes can be shares with other elements , each element class value should be unique within its scope. This uniqueness makes it a right choice for locating elements on a web page.

To locate an element by CLASS we use By.CLASS_NAME locator provided by the By class.

By is a class that provides a set of locating strategies to locate web elements on a web page.

Syntax:

```
element=driver.find_element(By.CLASS_NAME, "element_class_name")
```

```
# Python program to implement Locating by Class Name
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url
```

```
url = "https://www.sahyadri.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/"
```

```
driver.get(url)
```

```
# Replace ID_NAME with the Id of your element
```

```
try:
```

```
# Replace this with the id of the element
```

```
element = driver.find_element(By.CLASS_NAME, "entry-title")
```

except NoSuchElementException:

```
print("Not Found")
```

else:

```
print("Found")
```

3. Locating By Name

The name attribute of an HTML element is an identifier like a label . Unlike id ,class the name attribute doesn't necessarily unique within a web page. They are often used for naming form elements and is used in HTML forms to handle form controls such as input fields , radio button etc. The name attribute is essential for form submission and used to associate input data to specific name. This makes name a useful locator strategy especially when dealing with form submissions.

To locate an element by Name we use By.NAME locator provided by the By class.

By is a class that provides a set of locating strategies to locate web elements on a web page.

Syntax:

```
element=driver.find_element(By.NAME, "element_name")
```

```
# Python program to implement Locating by name
```

```
import time
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url

url = "https://www.sahyadri.org/about/contact-us/"

driver.get(url)

# Replace ID_NAME with the Id of your element

try:

    # Replace this with the name of the element

    element = driver.find_element(By.NAME, "feedback_area")

except NoSuchElementException:

    print("Not Found")

else:

    print("Found")
```

4. Locating By Tag Name

Every element in HTML document is identified by a TAG NAME. Tag Name are used to define the purpose and type of an element in a webpage. Tag Name lets us locate an element on a web page using the Tag Name like button , useful when we are looking for a specific element in our web page. Common HTML tag names are "div", "p", "input", "button" and many more.

Locating an element with tag name is useful specially when you want to interact with a group of elements that share a common tag name.

For example if we are testing a web application with multiple input fields , we can locate all the input fields by their "input" tag name ,this makes it easier to perform actions like filling out forms or validating inputs.

To locate an element by TAG NAME we use By.TAG_NAME locator provided by the By class.

By is a class that provides a set of locating strategies to locate web elements on a web page.

Syntax:

```
element=driver.find_element(By.TAG_NAME, "element_tag_name")
```

the Python program to implement Locating by tag name:

```
# Python program to implement Locating by tag name
```

```
import time
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url
```

```
url = "https://www.geeksforgeeks.org/about/contact-us/"
```

```
driver.get(url)
```

```
try:
```

```
    # Replace this with the Tag Name of the element
```

```
    element = driver.find_element(By.TAG_NAME, "select")
```

```
except NoSuchElementException:
```

```
    print("Not Found")
```

```
else:
```

```
    print("Found")
```

5. Locating By CSS Selectors

CSS Selectors are the patterns that used to select and style HTML elements in web development. CSS selectors allows us to target elements in a web page by their attributes ,position and hierarchy on a web page.

Some common CSS selectors are:

Element Selectors: Selects an element based on their HTML tag name. For example 'p' selects all <p> elements.

Class Selectors: Selects an element with a specific class attribute. For example '.btn' selects all elemets with 'class="btn"'.
(Note: The original text contains a typo 'elemets' which has been corrected to 'elements' in this transcription.)

ID Selector: Selects an element with specific id attribute . For example "#header" will select all the element with 'id="header"'.
(Note: The original text contains a typo 'the element with' which has been corrected to 'the element with' in this transcription.)

Descendent Selector: Selects an element that is a descendant of another element. For example, 'ul li' selects all '' element within a '' element

To locate an element by CSS Selector we use By.CSS_SELECTOR locator provided by the By class. By is a class that provides a set of locating strategies to locate web elements on a web page.

Syntax:

```
driver.find_element(By.CSS_SELECTOR, "element_css_selector")
```

```
# Python program to implement Locating by CSS Selector
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url
```

```
url = "https://www.geeksforgeeks.org/about/contact-us/"
```

```
driver.get(url)
```

```
try:
```

```
# Replace this with the css selector of the element
```

```
element = driver.find_element(By.CSS_SELECTOR, "div.title")
```

```
except NoSuchElementException:
```

```
    print("Not Found")
```

```
else:
```

```
    print("Found")
```

6. Locating By XPath

XPath (XML Path Language) is a powerful and one of the best locating strategy in Selenium for identifying elements on a web page. XPath is a language used for navigating and querying XML documents but its widely used in web scrapping and automation.

In selenium XPath is a valuable tool for selecting elements based on their attributes , position and relationship within the HTML structure of a web page.

XPath is a powerful strategy but its slower than other locators. However its a good choice when other locating strategy don't work or specially when the elements don't have reliable ID's, classes, or names.

While using XPath in Selenium we'll often inspect the webpage's HTML source code to identify the attributes that uniquely identify the element we want to interact.

These attributes can be id, class, or name.

XPath provides a powerful way to combine these attributes to pinpoint elements accurately.

Syntax-

```
element=driver.find_element(By.XPATH, "element_xpath")
```

```
# Python program of Locating by XPath
```

```
from selenium import webdriver
```

```
from selenium.common import NoSuchElementException
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
# Replace this with your actual url
```

```
url = "https://www.geeksforgeeks.org/about/contact-us/"
```

```
driver.get(url)
```

```
try:
```

```
    # Replace this with the XPATH selector of the element
```

```
    element = driver.find_element(By.XPATH,
```

```
                                   "/html/body/div[3]/div[2]/div[1]/div/div")
```

```
except NoSuchElementException:
```

```
    print("Not Found")
```

```
else:
```

```
    print("Found")
```